

Sprint Documentation

Sprint #25	Start Date: 01/09/2026	Final Date: 07/09/2026
Projetc Name:	civitas-documentation	
Ano: 5º	Análise e Desenvolvimento de Sistemas - AMS	
Team Members		
Adryan Thiago de Oliveira Francisco		
Carlos Alberto Ferreira Cândido		
Samuel Henrique Carneiro Pereira		
Yasmin Basso Moura		

Sprint Backlog

Task #	Description	Start Date	Final date
037	Capítulo 3. Modelagem UML do Sistema	01/09/2026	07/09/2026

Task Description

Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
037	Capítulo 3. Modelagem UML do Sistema	Adryan Thiago de Oliveira Francisco , Carlos Alberto Ferreira Cândido, Samuel Henrique Carneiro Pereira e Yasmin Basso Moura	Finish	10 horas	10 horas

Sprint Description

Objetivo

Revisar, adaptar e finalizar todo o **Capítulo 3 - Modelagem UML do Sistema**, garantindo que todos os diagramas, quadros e textos estejam relacionados exclusivamente ao sistema **Civitas** e sejam compatíveis com os requisitos, regras de negócio e código-fonte atual do projeto.

A documentação existente de projetos anteriores deverá ser utilizada como **referência de estrutura, organização e nível de detalhamento**. O objetivo não é simplesmente apagar tudo e começar novamente, mas aproveitar o modelo de construção das seções, quadros, explicações e diagramas, substituindo o conteúdo específico do projeto anterior por informações reais e originais do Civitas.

O capítulo deverá contemplar:

- **3.1 - Diagrama de Classes;**
- **3.2 - Dicionário de Classes;**
- **3.3 - Definição de Atores;**
- **3.4 - Casos de Uso;**
- **3.5 - Diagrama de Casos de Uso;**
- **3.6 - Diagrama de Casos de Uso - Individuais;**
- **3.7 - Diagrama de Sequência;**
- **3.8 - Diagrama de Atividade.**

Os tópicos **3.1**, **3.2** e **3.3** já possuem conteúdo desenvolvido para o Civitas, portanto deverão ser revisados e validados de acordo com a implementação atual.

A partir do tópico **3.4 - Casos de Uso**, existem conteúdos pertencentes a projetos anteriores. Esses conteúdos deverão ser utilizados como **modelo de como estruturar as respectivas seções**, porém todas as informações específicas deverão ser substituídas pelo contexto do Civitas.

3.1 - Diagrama de Classes

Revisar o Diagrama de Classes atualmente utilizado e garantir que:

- Todas as entidades representadas existam no código-fonte;
- Os atributos estejam atualizados;
- Os relacionamentos estejam corretos;
- As multiplicidades estejam representadas corretamente;
- As enumerações estejam compatíveis com o código;
- Não existam classes, atributos ou relacionamentos pertencentes a versões antigas do projeto.

Revisar também o texto explicativo do diagrama, mantendo o mesmo nível de detalhamento utilizado na documentação.

3.2 - Dicionário de Classes

Revisar todos os quadros do Dicionário de Classes e conferir:

- Nome das entidades;
- Nome dos atributos;
- Tipos dos atributos;
- Descrição de cada atributo;
- Relacionamentos;
- Coleções;
- Chaves estrangeiras;
- Enumerações.

Todas as informações deverão ser confrontadas com as entidades atuais presentes no código-fonte.

Utilizar os quadros já existentes como padrão visual e estrutural para manter a documentação uniforme.

3.3 - Definição de Atores

Revisar os atores existentes no Civitas e garantir que suas permissões estejam de acordo com o funcionamento real do sistema.

Atualmente deverão ser considerados os perfis existentes no projeto, como:

- Administrador;
- Funcionário;
- Visitante.

O texto deverá explicar claramente a função, responsabilidades e limitações de cada ator.

O Diagrama de Atores também deverá ser revisado.

3.4 - Casos de Uso

Desenvolver os Casos de Uso do **Civitas**.

A seção atualmente existente, pertencente a outro projeto, deverá ser utilizada como **referência de estrutura dos quadros e nível de detalhamento**.

Não copiar os casos de uso existentes. Utilizar somente a forma de apresentação como modelo.

Criar os quadros necessários para representar as funcionalidades disponíveis para cada ator do Civitas.

Os casos de uso deverão ser elaborados com base:

- Nos requisitos funcionais;
- Nas funcionalidades realmente implementadas;
- Nos níveis de acesso;
- Nas regras de negócio do Civitas.

Devem ser consideradas funcionalidades relacionadas a autenticação, usuários, secretarias, instituições, fornecedores, configurações, orçamentos, despesas, pagamentos, documentos, consultas, relatórios, auditoria, painel gerencial e perfil, conforme aplicável a cada ator.

3.5 - Diagrama de Casos de Uso

Desenvolver o **Diagrama Geral de Casos de Uso do Civitas**, representando graficamente as interações entre os atores e as funcionalidades do sistema.

Os diagramas existentes de outros projetos poderão ser utilizados como **referência visual e estrutural**, observando principalmente:

- Organização das funcionalidades;
- Distribuição dos atores;
- Legibilidade;
- Forma de apresentar os diagramas;
- Textos explicativos utilizados antes e depois das figuras.

O diagrama do Civitas deverá:

- Representar corretamente os atores;
- Mostrar as principais funcionalidades;
- Utilizar corretamente associações;
- Utilizar include, extend e generalização somente quando fizer sentido;
- Ser legível;
- Não apresentar funcionalidades inexistentes;
- Estar de acordo com os Casos de Uso descritos anteriormente.

Após o diagrama, elaborar um texto explicando sua estrutura e as principais interações representadas.

3.6 - Diagrama de Casos de Uso - Individuais

Desenvolver diagramas individuais para algumas das principais funcionalidades do Civitas.

Utilizar a documentação anterior como referência para compreender **como apresentar e detalhar um caso de uso individual**, mas desenvolver casos completamente novos para o Civitas.

Selecionar casos relevantes do sistema, priorizando operações importantes ou que possuam regras de negócio mais significativas.

Cada caso de uso individual deverá possuir:

- Ator responsável;
- Objetivo;
- Pré-condições;
- Fluxo principal;
- Fluxos alternativos, quando existirem;
- Pós-condições;
- Representação gráfica quando exigida pela estrutura da documentação.

Evitar criar diagramas individuais para operações excessivamente simples ou repetitivas.

3.7 - Diagrama de Sequência

Desenvolver Diagramas de Sequência para funcionalidades relevantes do Civitas.

Os diagramas existentes no documento poderão ser analisados para compreender a **estrutura esperada e o nível de detalhamento**, mas os participantes, mensagens, classes e operações deverão representar exclusivamente o Civitas.

Os diagramas deverão demonstrar corretamente a ordem das interações entre os componentes envolvidos durante a execução de uma operação.

Devem ser representados, quando aplicável:

- Usuário/ator;
- Interface;
- Controller;
- Service;
- Repository;

- Banco de dados;
- Demais componentes participantes do fluxo.

Priorizar funcionalidades que demonstrem adequadamente a arquitetura e as regras de negócio do sistema.

Após cada diagrama, explicar resumidamente o fluxo apresentado.

3.8 - Diagrama de Atividade

Desenvolver um ou mais Diagramas de Atividade representando processos relevantes do Civitas.

Utilizar os diagramas da documentação anterior somente como exemplo de **organização e representação UML**.

O diagrama deverá demonstrar:

- Início e fim do processo;
- Ações;
- Decisões;
- Condições;
- Fluxos alternativos;
- Retorno de operações;
- Sequência lógica das atividades.

Selecionar processos que possuam fluxo suficiente para justificar esse tipo de representação.

☑ Critérios de Aceitação

- Todo o Capítulo 3 deverá tratar exclusivamente do sistema Civitas;

Utilizar a documentação anterior como referência de estrutura e nível de detalhamento;

Não copiar textos, regras de negócio, atores, funcionalidades ou diagramas de outros projetos;

Preservar boas estruturas existentes quando elas puderem ser adaptadas para o Civitas;

Texto completamente original;

Plágio não será tolerado, inclusive utilizando documentação de anos anteriores;

Revisar os tópicos 3.1, 3.2 e 3.3 já existentes;

Desenvolver corretamente os tópicos 3.4, 3.5, 3.6, 3.7 e 3.8;

Garantir que todos os diagramas estejam compatíveis com o sistema atual;

Garantir que entidades, atributos, atores e funcionalidades estejam compatíveis com o código-fonte;

Utilizar corretamente a notação UML;

Todos os diagramas deverão possuir título, identificação e fonte;

Elaborar texto explicativo antes e/ou após os diagramas conforme o padrão da documentação;

Inserir referências bibliográficas quando necessário;
Seguir rigorosamente o padrão ABNT;
Revisar ortografia e gramática;
Verificar se todas as figuras utilizadas existem na pasta figs;
Verificar se todos os \label e \ref estão funcionando corretamente;
Compilar o documento sem erros.

✦ Observações Gerais

- A documentação do projeto anterior presente no arquivo é uma **excelente referência para entender como cada seção deve ser construída**;
- Utilizar essa documentação como modelo de **estrutura, organização, quantidade de informações, quadros, diagramas, legendas e textos explicativos**;
- O objetivo **não é apagar toda a estrutura existente e começar do zero**. Antes de remover uma parte, analisar como ela foi construída e recriar a mesma estrutura utilizando informações do Civitas;
- **Não copiar textos, dados, nomes, atores, funcionalidades, regras de negócio ou diagramas do projeto anterior**;
- Todo o conteúdo final deverá ser original e baseado no Civitas;
- A documentação anterior serve como **referência de formato e profundidade, e não como fonte de conteúdo**;
- Manter a padronização visual de todo o documento;
- Utilizar o código-fonte atual do Civitas como principal fonte para validar o funcionamento real do sistema;
- Diagramas existentes deverão ser conferidos antes de serem mantidos;
- Sempre que utilizar conceitos técnicos ou definições de UML, realizar a devida citação da fonte;
- Conferir se todas as referências utilizadas estão presentes em bibliografia.bib;
- As imagens dos diagramas deverão possuir boa resolução e permanecer legíveis após a compilação;
- Os arquivos das figuras deverão ser organizados corretamente na pasta figs;
- Evitar diagramas excessivamente grandes ou com informações desnecessárias;
- Conferir se os casos de uso possuem correspondência com os requisitos funcionais documentados anteriormente;
- Caso utilize Inteligência Artificial como ferramenta de apoio, **todo o conteúdo deverá ser revisado, adaptado ao contexto do projeto e validado antes da entrega**.

Sprint Results

O Capítulo 3 – Modelagem UML do Sistema foi revisado e estruturado integralmente com foco no sistema Civitas, contemplando os tópicos de 3.1 a 3.8. A revisão permitiu alinhar os

modelos UML à estrutura atual do projeto, evitando a permanência de conteúdos provenientes de documentações anteriores.

No Diagrama de Classes, foram revisadas as principais entidades do domínio, seus atributos, relacionamentos, multiplicidades e enumerações. A estrutura contempla entidades administrativas e financeiras, como Secretaria, Instituição, TipoInstituição, TipoDespesa, UnidadeMedida, Fornecedor, Orçamento, Despesa, Fluxo, Documento, Usuário e Auditoria, além das enumerações utilizadas pelo sistema.

O Dicionário de Classes foi organizado em quadros individuais, apresentando os atributos, tipos e respectivas descrições das entidades e enumerações. Também foram consideradas propriedades de navegação, coleções e chaves estrangeiras, mantendo a documentação compatível com a estrutura utilizada no backend do Civitas.

Na Definição de Atores, foram documentados os perfis de Administrador, Funcionário e Visitante, apresentando suas responsabilidades e limitações dentro do sistema. A organização dos atores foi utilizada como base para a elaboração dos casos de uso e dos respectivos diagramas.

Os Casos de Uso foram desenvolvidos e organizados de acordo com cada perfil de acesso, contemplando operações de autenticação, consultas, cadastros, atualizações, gerenciamento financeiro, unidades consumidoras e auditoria. Em seguida, foram estruturados os Diagramas de Casos de Uso, representando graficamente as principais interações entre os atores e as funcionalidades disponibilizadas pelo Civitas.

Também foi detalhado um Caso de Uso Individual relacionado ao cadastro de usuários, apresentando ator, objetivo, pré-condições, fluxo principal, fluxos alternativos e pós-condições. Para complementar a representação comportamental, foi desenvolvido o Diagrama de Sequência desse processo, demonstrando a comunicação entre a interface, UsuarioController, UsuarioService, UsuarioRepository e o banco de dados.

Por fim, o Diagrama de Atividade foi elaborado para representar o processo de autenticação, demonstrando o fluxo desde a inserção das credenciais pelo usuário até a validação dos dados e o direcionamento para o sistema ou a realização de uma nova tentativa em caso de informações inválidas.

Dessa forma, o capítulo passou a apresentar uma visão estruturada da arquitetura, dos elementos de domínio, dos atores e dos principais comportamentos do Civitas, mantendo a documentação alinhada à implementação atual e seguindo o padrão de organização e apresentação definido para o projeto.

Assinaturas